

TABLE OF CONTENTS

1. ABSTRACT	2
2. INTRODUCTION	3
3. OBJECTIVES.....	4
4. SCOPE OF THE PROJECT	4
5. LITERATURE SURVEY	5
6. SYSTEM OVERVIEW	6
7. COMPONENTS USED	7
8. HARDWARE ARCHITECTURE.....	8
9. SOFTWARE ARCHITECTURE.....	9
10. METHODOLOGY.....	10
11. ROBOT OPERATING SYSTEM (ROS) – DETAILED	11
12. LIDAR SENSOR WORKING AND INTEGRATION WITH ROS	12
13. ULTRASONIC SENSOR-BASED OBSTACLE DETECTION.....	13
14. SENSOR OPTIMIZATION TECHNIQUES.....	14
15. NAVIGATION AND PATH PLANNING.....	16
16. IMPLEMENTATION USING RASPBERRY PI 4 MODEL B.....	17
17. PROGRAMMING CODE	18
18. TESTING AND RESULTS.....	21
19. ADVANTAGES AND LIMITATIONS	22
20. FUTURE ENHANCEMENTS	23
21. ROBOT GALLERY	23
22. TEAM CONTRIBUTIONS.....	25
23 CONCLUSION / SUMMARY.....	25
24. REFERENCES	26

1. ABSTRACT

Autonomous navigation has become a critical area in robotics, especially in applications such as industrial automation, warehouse logistics, and service robotics. This project focuses on the design and development of an autonomous mobile robot that utilizes a LiDAR sensor integrated with a Raspberry Pi 4 Model B for real-time navigation and obstacle detection. Unlike traditional systems that depend heavily on high-end computing systems, this project demonstrates that efficient autonomous navigation can be achieved using a compact and cost-effective embedded system.

The robot employs LiDAR technology for accurate mapping and localization, while ultrasonic sensors provide additional redundancy for short-range obstacle detection. The integration of the Robot Operating System (ROS) enables modular development, real-time communication, and scalability. ROS plays a crucial role in processing LiDAR data, generating maps, and enabling autonomous decision-making.

The project also emphasizes sensor optimization techniques to improve performance under varying environmental conditions. Data fusion between LiDAR and ultrasonic sensors ensures higher reliability and accuracy in obstacle detection. The navigation system is designed to handle dynamic environments, making the robot suitable for real-world applications.

2. INTRODUCTION

The field of robotics has undergone a transformative evolution over the past few decades, shifting from repetitive, pre-programmed industrial arms to intelligent, autonomous systems capable of navigating complex and dynamic environments. At the heart of this evolution lies the challenge of autonomous navigation—the ability of a robot to move from one point to another without human intervention, while simultaneously avoiding obstacles and optimizing its path. This project, "Autonomous Mobile Navigation Bot Using LiDAR Sensor with Raspberry Pi 4," addresses this challenge by designing a robust, cost-effective robotic platform tailored for indoor navigation.

Traditionally, autonomous mobile robots (AMRs) relied on expensive hardware and high-performance computing units, such as industrial PCs or NVIDIA Jetson boards, to process heavy sensor data. While effective, these solutions often come with high power consumption and prohibitive costs, making them inaccessible for small-scale research, educational purposes, or startups. However, the recent surge in performance of single-board computers (SBCs), specifically the Raspberry Pi 4, has bridged this gap. This project capitalizes on the accessibility of the Raspberry Pi 4, proving that capable autonomous navigation is no longer confined to high-budget laboratories.

The core of our navigation system relies on Light Detection and Ranging (LiDAR) technology. Unlike simple infrared or ultrasonic sensors that only provide distance measurements in a single direction, a LiDAR sensor spins rapidly, emitting laser pulses to generate a 360-degree point cloud of the surrounding environment. This data allows the robot to "see" the world in detail, enabling it to distinguish between walls, furniture, and open spaces. By integrating this LiDAR with the Robot Operating System (ROS), we create a software ecosystem that handles complex tasks like Simultaneous Localization and Mapping (SLAM) and path planning with relative ease.

Furthermore, this project does not rely solely on LiDAR. To ensure safety and redundancy, we have integrated ultrasonic sensors into the system. These sensors act as a safety net for close-range obstacle detection, specifically for blind spots that the LiDAR might miss due to its mounting height or angular resolution. By combining these disparate sensing modalities—LiDAR for long-range environmental mapping and ultrasonics for immediate proximity alerts—we create a multi-layered perception system. This report documents the complete journey of this project, from the theoretical underpinnings of ROS and sensor fusion to the practical challenges of hardware integration and algorithm implementation.

3. OBJECTIVES

- Design an autonomous mobile robot using LiDAR.
- Implement ROS (Robot Operating System) on Raspberry Pi 4.
- Achieve real-time obstacle detection and avoidance.
- Integrate ultrasonic sensors for redundancy and safety.
- Optimize sensor performance for accuracy in various environments.
- Develop efficient navigation and mapping algorithms using SLAM.

4. SCOPE OF THE PROJECT

The scope of this project is limited to indoor navigation environments such as homes, offices, or small warehouses. The robot is designed to navigate flat surfaces and handle static obstacles (walls, furniture) as well as dynamic obstacles (humans walking by). The project utilizes 2D LiDAR, meaning it creates a map of the floor plan but does not detect overhanging obstacles (like table edges) without the aid of the supplementary ultrasonic sensors. The software scope includes implementing the ROS Melodic or Noetic navigation stack, Mapping for SLAM, and Python-based scripts for sensor interfacing.

5. LITERATURE SURVEY

The development of autonomous navigation systems has been extensively documented in academic literature, with approaches ranging from simple line-following robots to complex AI-driven vehicles. In the early stages of mobile robotics, navigation was primarily achieved using dead reckoning and line-following algorithms. These systems were fundamentally limited by their lack of awareness regarding the surrounding environment; a robot could follow a line but could not detect an obstacle blocking that line or navigate off the predetermined path. As sensor technology improved, researchers began integrating infrared (IR) sensors and ultrasonic rangefinders to introduce basic obstacle avoidance capabilities.

A significant milestone in robotic navigation was the introduction of Simultaneous Localization and Mapping (SLAM). Pioneering work in the 1990s and 2000s demonstrated that robots could construct a map of an unknown environment while simultaneously keeping track of their location within it. Early implementations of SLAM relied heavily on sonar arrays, which suffered from poor angular resolution and susceptibility to noise. The advent of vision-based SLAM (vSLAM) using cameras offered rich data but introduced intense computational loads and challenges related to lighting conditions. Robots often struggled to navigate in low-light environments or places with visually featureless walls.

In recent years, LiDAR technology has become the gold standard for 2D indoor navigation. Research papers comparing LiDAR-based SLAM against camera-based methods consistently highlight LiDAR's superior accuracy in distance measurement and its independence from ambient lighting. However, much of the existing literature focuses on high-grade LiDAR sensors (such as those from SICK or Hokuyo) paired with expensive industrial computers. There is a growing body of research exploring the "democratization" of robotics, utilizing cost-effective sensors like the RPLiDAR series and consumer-grade SBCs.

Studies focusing on Raspberry Pi in robotics highlight its sufficiency for running lightweight Linux distributions and ROS, although they often note limitations in processing high-frequency 3D point clouds. This project aligns with this niche of research. By reviewing the literature, it becomes evident that while high-end systems are well-documented, there is a need for comprehensive, reproducible models that utilize the Raspberry Pi 4's upgraded specs (specifically its 4GB/8GB RAM and faster CPU clock speed) to handle 2D LiDAR data effectively. Furthermore, literature on sensor fusion suggests that while LiDAR is excellent for structural mapping, it can miss thin objects or obstacles below its scan plane. Integrating ultrasonic sensors fills these gaps, a hybrid approach strongly supported by recent studies on robust indoor navigation systems.

6. SYSTEM OVERVIEW

The autonomous navigation bot is a complex integration of hardware and software designed to mimic human perception and decision-making. To understand the system, it is best to break it down into three distinct subsystems: the Perception System, the Processing System, and the Actuation System. These three loops work in a continuous cycle—Sense, Think, Act—at a rate that allows for real-time navigation.

The Perception System acts as the robot's eyes. The primary sensor is the LiDAR, which is mounted at a specific height to scan the horizontal plane of the room. As it rotates, it captures hundreds of data points per second, measuring the distance to walls, chairs, and other obstacles. This data is fed to the Raspberry Pi as a "Laser Scan" message. Complementing the LiDAR are the ultrasonic sensors (HC-SR04). These are placed at the front and corners of the robot to detect obstacles that might be invisible to the LiDAR, such as table legs or low-lying objects. While the LiDAR provides a detailed 360-degree map, the ultrasonics provide immediate, short-range "stop" signals.

The Processing System is the brain of the operation, hosted on the Raspberry Pi 4 Model B. Here, the Robot Operating System (ROS) acts as the middleware. The raw data from the sensors is not useful on its own; it must be processed. ROS nodes running on the Pi take the LiDAR data to build a grid map of the room using a SLAM algorithm (such as Gmapping). The system also runs a navigation stack (like Move Base). This stack takes the map and the robot's current goal (e.g., "go to coordinates X, Y") and calculates a viable path. It constantly checks the sensor data against the planned path. If a new obstacle appears, the replanning algorithm kicks in instantly to find a new route around the obstruction.

The Actuation System translates the digital decisions of the processor into physical movement. This subsystem consists of the motor driver (L298N), the DC geared motors, and the wheels. When the navigation stack decides the robot needs to turn left or move forward, it publishes a "Twist" message containing linear and velocity commands. A ROS node listening to these commands converts them into Pulse Width Modulation (PWM) signals. The motor driver receives these PWM signals to adjust the voltage going to the left and right motors, thereby steering the robot. Power management units ensure that the sensors and processor receive stable voltage while the motors draw variable current during acceleration and turns.

7. COMPONENTS USED

Hardware:

- Raspberry Pi 4 Model B: The central computing unit. Features a Quad-core Cortex-A72 (ARM v8) 64-bit SoC running at 1.5GHz. Provides the necessary processing power to run Linux and ROS Noetic smoothly.
- LiDAR Sensor (RPLiDAR A1/A2): A 360-degree 2D laser scanner. Measures distances by calculating the time of flight of laser pulses, creating a detailed point cloud for mapping.
- Ultrasonic Sensors (HC-SR04 ×2): Provide distance measurements via sonar for short-range obstacle detection and redundancy.
- Motor Driver (L298N): A dual H-bridge motor driver allowing control of DC motor speed and direction via PWM signals.
- DC Motors: Geared motors providing the necessary torque for movement.
- Chassis: The physical frame holding all components.
- Power Supply: High-capacity Lithium-Polymer (LiPo) battery pack (2S or 3S).

Software:

- ROS (Robot Operating System): Middleware for robotic communication and control.
- Python: Programming language for custom sensor nodes.
- Linux (Ubuntu for Raspberry Pi): The underlying operating system.

8. HARDWARE ARCHITECTURE

The hardware architecture of the autonomous navigation bot is designed as a layered system, separating power management, computation, perception, and actuation into distinct but interconnected blocks. This separation ensures that noise from the motors does not interfere with the sensitive sensors, and that the processing unit receives stable power to prevent system crashes during critical navigation tasks.

The Power Management Unit: The foundation of the architecture is the power system. The robot is powered by a high-discharge Lithium-Polymer (LiPo) battery. This raw battery voltage is split into two distinct rails. The first rail goes directly to the motor driver. Motors are noisy devices; they generate back Electromotive Force (EMF) and voltage spikes during operation. By powering them directly from the battery, we isolate this electrical noise from the rest of the circuit. The second rail passes through a DC-DC Buck Converter. This component steps down the variable battery voltage to a steady 5V, which is then used to power the Raspberry Pi and the sensors. A stable 5V rail is crucial; if the voltage dips when the motors start (a phenomenon called voltage sag), the Raspberry Pi could brownout and reset, causing the robot to lose control.

The Central Processing Hub: The Raspberry Pi 4 acts as the central nervous system. It connects to the peripherals via USB and General-Purpose Input/Output (GPIO) pins. The LiDAR sensor connects via USB, which provides both power and a high-speed data interface. This allows the LiDAR to stream scan data without overwhelming the Pi's CPU. The ultrasonic sensors connect to the GPIO pins, utilizing trigger and echo pins to measure distance. The GPIO pins are also used to send Pulse Width Modulation (PWM) signals to the motor driver. Physically, the architecture ensures the Raspberry Pi is mounted centrally, minimizing wire lengths to the sensors and motors, which reduces signal interference and cable tangling.

Actuation Circuitry: The L298N motor driver serves as the interface between the low-voltage logic of the Pi (3.3V) and the higher-voltage motors. The architecture uses a differential drive configuration, meaning there are two driven wheels and one caster wheel. The L298N receives two sets of logic signals from the Pi: one set for direction (digital High/Low) and one set for speed (PWM). The driver board contains two H-Bridges. An H-Bridge is an arrangement of four transistors that allows the circuit to reverse the polarity of the voltage going to the motors. This capability is essential for the robot to move backward or spin in place. The mechanical assembly places the motors on the sides of the chassis, providing a wide wheelbase for stability during turns.

9. SOFTWARE ARCHITECTURE

While the hardware provides the body of the robot, the software architecture provides the intelligence. The software is built upon the Robot Operating System (ROS), running on top of Ubuntu Linux. This architecture is modular, meaning different functionalities—like sensing, mapping, and driving—are handled by separate "Nodes" that communicate with each other. This modularity makes the system easier to debug and upgrade.

The ROS Master and Nodes: At the core of the software architecture is the ROS Master. Think of the Master as a traffic controller or a bulletin board. All software nodes must register with the Master to communicate. Our system consists of several key nodes. The `rplidarNode` is responsible for communicating with the LiDAR hardware. It takes the raw serial data from the USB port and converts it into a standardized ROS message called Laser Scan. The ultrasonic node is a custom Python script that reads the GPIO pins, calculates distance based on the speed of sound, and publishes this data as a Range message. The `motor_driver_node` subscribes to command messages and translates them into specific PWM signals for the L298N driver.

The Navigation Stack: The most complex part of the software architecture is the navigation stack. This is a pre-built suite of ROS tools that we configure for our robot. It consists of two main parts: the Global Planner and the Local Planner.

- **Global Planner:** This uses the map of the room (generated by SLAM) to calculate the best path from the robot's current position to the goal. It uses algorithms like Dijkstra's or A* (A-Star) to find the shortest path that avoids static obstacles like walls.
- **Local Planner:** This handles immediate movement. It looks at the Global Planner's path and the current sensor data (obstacles right in front of the robot) to calculate velocity commands. It uses algorithms like Dynamic Window Approach (DWA) to ensure the robot doesn't hit a chair that wasn't on the map, or to avoid a person walking in front of it.

Transform System (TF): A critical, often invisible part of the software architecture is the TF (Transform) tree. The robot needs to know where its sensors are relative to its centre (base link). For example, the LiDAR is mounted 5cm above the centre. The TF tree broadcasts this relationship constantly. When the Navigation Stack plans a path, it uses the TF tree to translate the world coordinates into the LiDAR's coordinate frame. This ensures that if the robot turns right, the map rotates correctly in the robot's "mind."

10. METHODOLOGY

The development of the autonomous navigation bot follows a systematic, phased methodology. This approach ensures that each layer of the system is validated before moving to the next, reducing the complexity of debugging an integrated system.

Phase 1: Environment Setup and OS Configuration: The first step involves preparing the Raspberry Pi. We install a 64-bit version of Ubuntu Server or Raspberry Pi OS Lite to keep the system lightweight. Since the Raspberry Pi 4 has limited cooling compared to a desktop PC, we configure the CPU governor to prioritize performance. We then install ROS Noetic Nanjemoy's, the version of ROS compatible with Ubuntu 20.04. This involves setting up the repositories, keys, and core libraries. Once ROS is installed, we create a "Catkin Workspace," a directory structure where our custom code and downloaded packages will reside.

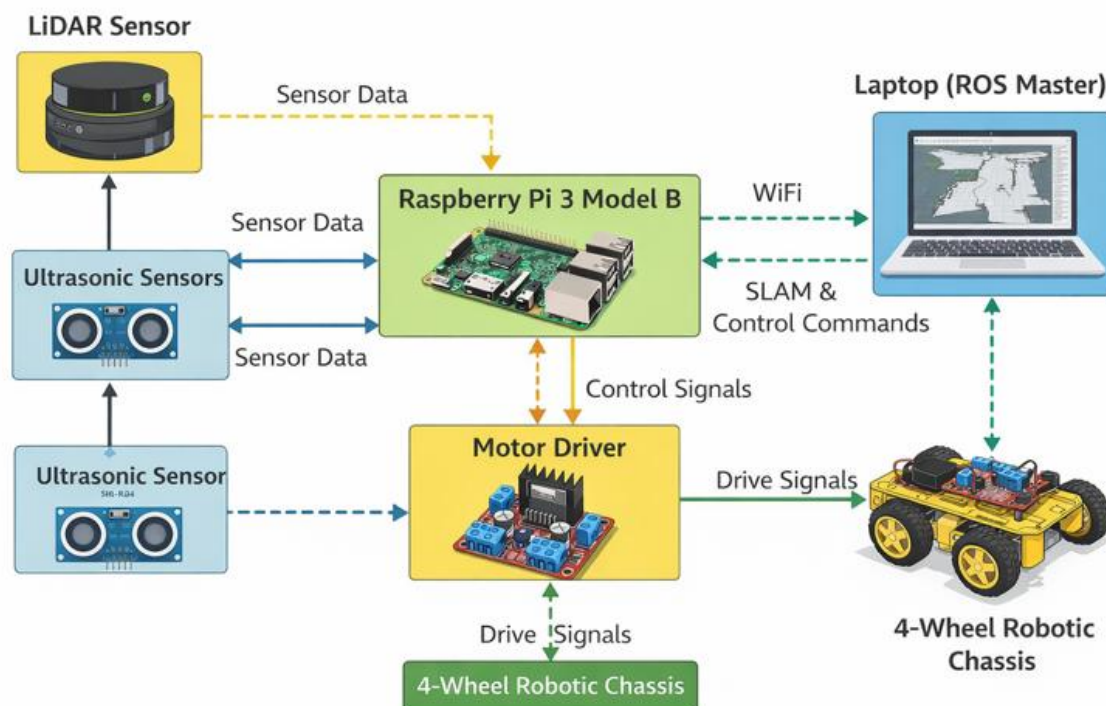
Phase 2: Hardware Integration and Unit Testing: Before writing complex navigation code, we verify the hardware. We write simple Python scripts to test the ultrasonic sensors individually, ensuring they return accurate distance readings within a 2cm to 400cm range. We then test the DC motors using the GPIO pins to ensure they spin forward and backward correctly with the L298N driver. The LiDAR is connected via USB, and we use the manufacturer's provided driver to launch a test scan. We visualize this data in Rviz to ensure the LiDAR is producing a clean 360-degree scan without missing sectors or "ghost" points caused by reflective surfaces.

Phase 3: SLAM and Map Generation: With sensors working, we proceed to Simultaneous Localization and Mapping (SLAM). We launch the mapping node. Initially, the robot assumes it is in a void at coordinate (0,0). Using a joystick (teleoperation), we manually drive the robot around the test area. As the robot moves, the LiDAR detects walls, and the mapping algorithm aligns these scans to build a grid map. This phase is iterative; if the robot moves too fast, the wheels might slip, causing the map to skew. We carefully navigate the robot through the entire environment, ensuring all rooms and hallways are captured. The result is a .pgm file (a pixel-based map) and a .yaml configuration file that defines the map's resolution and origin.

Phase 4: Localization and Navigation Tuning: Once the map is built, we switch to navigation mode. We use the AMCL (Adaptive Monte Carlo Localization) package. AMCL uses a particle filter to determine the robot's position within the known map by comparing the current LiDAR scan to the saved map data. We then configure the move base navigation stack. This requires tuning "Cost maps"—dynamic layers added over the static map to represent obstacles the robot is currently seeing. We must tune parameters like the "inflation radius" (how much space the robot tries to keep clear from obstacles) and the "min/max velocity" to ensure smooth, non-oscillatory movement.

Phase 5: Autonomous Operation and Fusion: In the final phase, we test the full autonomy. We set a 2D Nav Goal in Rviz (clicking a point on the map). The robot calculates a path and begins driving. During this phase, we introduce dynamic obstacles (like a person walking in front of it) to test the sensor fusion. We expect the LiDAR to detect the obstacle and the local planner

to stop or replan. We also verify that if an object gets within 20cm of the ultrasonic sensors, a safety override kicks in to halt the motors, acting as a final "emergency stop."



11. ROBOT OPERATING SYSTEM (ROS) – DETAILED

The Robot Operating System (ROS) is the backbone of this project. Despite its name, ROS is not an actual operating system like Windows or Linux; rather, it is a middleware—a collection of software frameworks that sit on top of a host OS to manage robot complexity. ROS provides standard services to a heterogeneous computer cluster, such as hardware abstraction, low-level device control, implementation of commonly used functionality, message-passing between processes, and package management.

Nodes and the Master: The fundamental computational unit in ROS is the "Node." A node is essentially a process that performs computation. In our project, the Python script that reads the ultrasonic sensor is a node. The C++ program that controls the motors is a different node. ROS is designed to be distributed; these nodes could be running on the Raspberry Pi, or on a laptop connected via Wi-Fi. To coordinate these nodes, ROS uses a "Master." The Master allows nodes to find each other. When the ultrasonic node starts up, it registers with the Master and says, "I am publishing distance data." When the navigation node starts, it asks the Master, "Is anyone publishing distance data?" The Master facilitates this handshake.

Topics, Publishers, and Subscribers: Nodes communicate primarily via "Topics." A Topic is a named bus over which messages are exchanged. This communication follows a

"Publish/Subscribe" model, which is a decoupled method of communication. The Publisher (e.g., the LiDAR node) does not need to know who is reading the data. It simply pushes data to the /scan topic. The Subscriber (e.g., the Mapping node) does not need to know who produced the data; it just listens to /scan. This is incredibly useful for robotics because we can easily add new sensors or recording tools just by subscribing to existing topics without rewriting the sensor code.

Messages and Services: Data traveling over topics is structured as "Messages." For example, the Laser Scan message is a complex structure containing an array of floats representing distances, an array of angles, and timestamp information. This standardization means any ROS-compatible tool can understand LiDAR data. For request-response communication (where a node needs an immediate answer), ROS uses "Services." For instance, we might use a service to "Clear the Cost map" or "Save the Map," where the robot sends a request and waits for a confirmation before proceeding.

TF (Transform Library): Perhaps the most powerful yet complex concept in ROS is the TF library. In robotics, coordinate frames are everywhere. We have a base link (center of robot), laser frame (where the LiDAR is), Odom frame (odometry position), and map frame (global position). TF keeps track of the relationship between all these frames over time. If the robot rotates 10 degrees, the TF tree updates the relationship between laser_frame and base_link. This allows the navigation stack to seamlessly transform a point seen by the LiDAR into a point on the static map, a requirement that is mathematically intensive but handled automatically by ROS tools.

12. LIDAR SENSOR WORKING AND INTEGRATION WITH ROS

Light Detection and Ranging, or LiDAR, functions as the primary eye of the autonomous robot. Its operating principle is rooted in the Time of Flight (ToF) concept. The sensor emits a pulsed laser beam that travels at the speed of light. When this beam strikes an object—a wall, a chair, or a person—a portion of the light is reflected back to the sensor's receiver. The LiDAR measures the exact time interval between the emission of the pulse and the detection of its reflection. Since the speed of light is a known constant, the distance can be calculated with high precision using the formula: $\text{Distance} = (\text{Speed of Light} \times \text{Time of Flight}) / 2$.

In our specific setup, we utilize a spinning LiDAR (such as the RPLiDAR A1). This sensor consists of a laser module and a receiver mounted on a motor that spins at a configurable speed (typically 5Hz to 10Hz). As it spins, it captures distance measurements at angular increments (often fractions of a degree). This generates a 360-degree profile of the environment, producing a dataset known as a "point cloud." Each point in this cloud represents a specific coordinate (x, y) relative to the robot's center.

Integration with ROS: The raw output from the LiDAR sensor is a stream of hexadecimal data transmitted via a serial connection (USB). The Raspberry Pi cannot use this raw data directly

for navigation; it must be translated into a standard format that the Robot Operating System understands. This is achieved using the `rplidar_ros` package. This package contains a ROS node (written in C++ or Python) that acts as a driver.

When we launch the LiDAR driver, it opens the serial port (e.g., `/dev/ttyUSB0`) at a specific baud rate. It reads the raw scan data, parses the angles and distances, and publishes them to a ROS topic named `/scan`. The message type used is `sensors/Laser Scan`. This message is a standardized array containing the ranges (distances) and intensities. By publishing to `/scan`, any other node in the system—such as the SLAM mapping node or the obstacle avoidance node—can subscribe to this topic and "see" what the LiDAR sees. This decoupling is vital; the navigation algorithm doesn't need to know the intricacies of the RPLiDAR hardware protocol, it simply consumes standard Laser Scan messages.

13. ULTRASONIC SENSOR-BASED OBSTACLE DETECTION

While LiDAR provides high-fidelity mapping, it has limitations: it can fail to detect transparent surfaces like glass, and it has a minimum range (blind spot) immediately in front of the sensor. To compensate for this, we integrate HC-SR04 ultrasonic sensors as a redundant safety layer. These sensors operate on the principle of echolocation, similar to how bats or dolphins navigate.

Working Principle: The HC-SR04 has two main components: a Trigger pin and an Echo pin. To initiate a measurement, the Raspberry Pi sends a 10-microsecond High pulse to the Trigger pin. This causes the sensor to emit eight cycles of a 40kHz ultrasonic burst. These sound waves travel through the air and reflect off the nearest object. The Echo pin then goes High, and the timer starts. It remains High until the sensor detects the reflected sound wave, at which point it goes Low.

Calculation: The distance is calculated by measuring the duration the Echo pin stayed High. The formula is: $\text{Distance} = (\text{Duration} \times \text{Speed of Sound}) / 2$. Since the speed of sound in air is approximately 343 meters per second (or 0.0343 cm/ μ s), we can compute the distance in centimetres. Division by two is necessary because the sound travelled to the object and back, so we only need the one-way distance.

Implementation Logic: In our robot's software architecture, we run a dedicated Python script that interacts with the Raspberry Pi's General-Purpose Input/Output (GPIO) pins. The script uses a while loop to continuously poll the sensors. We utilize two ultrasonic sensors: one facing the front-left and one facing the front-right. This "stereo" setup helps detect obstacles that might be in the path of the wheels during a turn.

The logic involves a threshold check. If the calculated distance from either sensor drops below a safety threshold (e.g., 20 cm), the system triggers an interrupt. This interrupt overrides the

standard navigation commands coming from the ROS path planner. Effectively, it forces the robot to stop immediately, regardless of what the LiDAR sees.

14. SENSOR OPTIMIZATION TECHNIQUES

In real-world robotics, raw sensor data is rarely perfect. It is often plagued by noise—random fluctuations in readings that do not correspond to real objects. If the navigation computer takes this noisy data literally, the robot might jitter, stop for non-existent obstacles, or crash through a wall because the data was filtered out. To ensure smooth and reliable autonomous navigation, we must apply specific optimization techniques to the data collected from both the LiDAR and Ultrasonic sensors.

1. Range Filtering (Outlier Removal): The first line of defence is simple range filtering. Both sensors have physical limitations. For instance, the LiDAR cannot measure distances closer than 0.15 meters (its blind spot), and the ultrasonic sensor has a maximum range of about 4 meters. Any reading outside these bounds is physically impossible or unreliable. In our ROS configuration (yaml files), we set `min_range` and `max_range` parameters. If a sensor returns a value of 0.0 meters (often caused by signal interference) or 30 meters (beyond the LiDAR's capability), the software discards this value immediately before it reaches the navigation stack.

2. Moving Average Filter: Ultrasonic sensors are particularly susceptible to "jitter." A single reading might fluctuate between 50cm and 55cm rapidly, causing the robot's logic to oscillate. To smooth this out, we implement a Moving Average filter. Instead of taking the instantaneous reading, the robot stores the last 5-10 readings in memory and calculates the average. These smoothest out sudden spikes and provides a stable distance metric for the logic controller.

3. Median Filter: Occasionally, sensors produce "spike noise"—a single reading that is drastically different from reality (e.g., reading 300cm when the object is actually 30cm away). A simple average filter would be skewed by this spike. A median filter solves this by taking a window of readings, sorting them, and picking the middle value. This effectively removes outliers that are significantly higher or lower than the actual distance.

4. Ray Casting and Inflation (ROS Cost maps): In the ROS navigation stack, we optimize the LiDAR data for safety using "ray casting" and "inflation." The navigation stack generates a "cost map"—a grid where each cell has a value representing the likelihood of an obstacle. We configure an "inflation radius." If the LiDAR detects a wall, the robot marks the cells corresponding to the wall as "lethal" (no-go zones). It then marks the surrounding cells (within the inflation radius, e.g., 20cm) as "costly." The path planner will then try to keep the robot as far away from the wall as possible, rather than driving right alongside it. This optimization prevents the sides of the robot from clipping obstacles due to minor localization errors.

5. Computational Throttling: The Raspberry Pi 4 is powerful, but not infinite. Processing LiDAR data at 30Hz can consume high CPU usage. We optimize by throttling the scan data.

While the LiDAR spins at 10Hz, we might configure the navigation stack to only process updates at 5Hz. This frees up CPU resources for other critical tasks like path planning and motor control, ensuring the robot remains responsive rather than lagging due to computational overload.



15. NAVIGATION AND PATH PLANNING

Navigation is the cognitive capability that transforms a mobile robot from a remote-controlled toy into an intelligent machine. It is the process of determining a viable path from a starting location to a goal location while avoiding obstacles. In our project, navigation is handled by the ROS move base stack, which divides this complex problem into two distinct layers: Global Planning and Local Planning.

Global Path Planning: The Global Planner operates on the static map we generated using SLAM. It treats the environment as a known grid of free space and occupied space. When the user sets a 2D Nav Goal in Rviz, the Global Planner calculates the most efficient geometric path to that destination. We primarily use the A* (A-Star) algorithm, a popular graph traversal and path search algorithm. It works by maintaining a tree of paths originating at the robot's start position and extending those paths one edge at a time until the goal is reached. It uses a "cost function" to prioritize paths that minimize distance while avoiding known static obstacles like walls. This global path is relatively static; it doesn't change unless the map changes.

Local Path Planning (Obstacle Avoidance): While the Global Planner figures out *where* to go, the Local Planner figures out *how* to drive there in real-time. This is where dynamic obstacle avoidance happens. The world is not static; chairs move, people walk, and the robot's wheels might slip. The Local Planner takes the geometric path from the Global Planner and breaks it down into small, incremental velocity commands (linear velocity x and angular velocity θ). We utilize the Dynamic Window Approach (DWA). DWA works by sampling a finite window of velocity pairs (forward speed + turning speed) that the robot is physically capable of reaching in the next time interval. For each sampled velocity, it simulates a short trajectory. It then scores these trajectories based on three criteria: how close it gets to the global path (progress), how far it is from obstacles (clearance), and how fast it goes (velocity). The trajectory with the highest score is executed.

Cost maps and Inflation: A critical concept in our navigation system is the "Costmap." The robot maintains two cost maps: a Global Costmap (large, covers the whole map) and a Local Costmap (small, covers immediate surroundings). The Costmap is a grid where each cell has a value from 0 to 254. A value of 0 is free space, while 254 is a lethal obstacle. We configure an "inflation radius." For example, if the inflation radius is 20cm, the cost values increase as cells get closer to an obstacle. This creates a "repulsive force" field, ensuring the robot prefers to stay in the centre of a hallway rather than scraping the walls. If the ultrasonic sensors detect a close-range obstacle, they inject a lethal obstacle directly into the Local Costmap, forcing the Local Planner to stop or reroute instantly.

16. IMPLEMENTATION USING RASPBERRY PI 4 MODEL B

The implementation phase involves bringing the theoretical design to life on the Raspberry Pi 4 Model B, serving as the embedded brain of the robot. The choice of the Raspberry Pi 4 is pivotal because of its quad-core Cortex-A72 architecture, which offers a significant performance boost over previous models, allowing it to handle the computationally intensive tasks of ROS and SLAM.

Operating System and ROS Setup: We deploy the Ubuntu Server 20.04 LTS (64-bit) operating system on the Raspberry Pi. While the standard Raspberry Pi OS is user-friendly, Ubuntu is the standard for ROS Noetic (the last ROS version designed for Python 2/3 compatibility on standard Linux). Using a "headless" setup (without a monitor connected to the Pi), we access the system remotely via Secure Shell (SSH). This reduces the load on the Pi's GPU, freeing up resources for the CPU. The installation process involves adding the ROS repository to Ubuntu's sources list, adding the authentication keys, and running `apt install ros-noetic-desktop-full`. This installs the core libraries, the visualization tool (Rviz), the Gazebo simulator, and the navigation stack.

Hardware Interfacing: The physical implementation requires careful pin mapping. The LiDAR is connected via one of the Pi's USB 3.0 ports (blue connectors) to ensure high bandwidth data transfer. The L298N motor driver is connected to the GPIO pins; we use specific pins for Pulse Width Modulation (PWM) output to control speed, and standard digital pins to control the direction (Forward/Reverse) of the motors. The ultrasonic sensors require precise timing. We utilize the GPIO library in Python to toggle the Trigger pins and measure the duration of the Echo pins. To ensure accurate timing for the ultrasonic sensors without lag, we implement the code using "busy waiting" loops or real-time kernel patches to minimize the operating system's scheduling delays.

Power and Thermal Management: A critical practical aspect of implementation is managing power and heat. The Raspberry Pi 4 is notorious for thermal throttling; if the CPU temperature exceeds 80°C, it automatically underclocks itself, causing navigation to stutter. To prevent this, the implementation includes an active cooling solution—a USB fan or a copper heatsink case—to keep the processor cool under load. Power-wise, the implementation requires a clean power supply. The motors draw large currents, causing voltage dips. We implement a hardware solution using a high-quality Buck Converter to provide a stable 5V line to the Raspberry Pi, isolating it from the noisy motor power rail.

17. PROGRAMMING CODE

A. ULTRASONIC OBSTACLE DETECTION MODULE

python

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

TRIG = 5
ECHO = 6

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

def get_distance():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    distance = (pulse_end - pulse_start) * 17150
    return distance

try:
    while True:
        distance = get_distance()
        print(f"Obstacle Distance: {distance:.2f} cm")

        if distance < 20:
            print("Obstacle Detected - Stop Condition Triggered")
        else:
            print("Path Clear")

        time.sleep(0.5)

except KeyboardInterrupt:
    GPIO.cleanup()
```

B. ROS SIMULATED LIDAR PUBLISHER FOR MOBILE ROBOT

Python

```
import rclpy

from rclpy.node import Node

from sensor_msgs.msg import LaserScan
import math

import random

class FakeLidarPublisher(Node):
    def __init__(self):

        super().__init__('fake_lidar_publisher')

        self.publisher_ = self.create_publisher(LaserScan, '/scan', 10)
        self.timer = self.create_timer(0.1,
            self.publish_scan)self.get_logger().info("Simulated LiDAR started") def
        publish_scan(self):

            scan = LaserScan()

            scan.header.frame_id = "laser_frame"
            scan.angle_min = -math.pi
            scan.angle_max = math.pi
            scan.angle_increment = math.pi / 180
            scan.time_increment = 0.0

            scan.scan_time = 0.1

            scan.range_min = 0.12

            scan.range_max = 8.0

            ranges = []

            for i in range(360):

                angle = math.radians(i)

                distance = 3.0 + 0.5 * math.sin(angle * 4)
                distance += random.uniform(-0.05, 0.05)

                if distance < 0.2:
                    distance = 0.2

                if distance > 8.0:
                    distance = 8.0

                ranges.append(distance)

            scan.ranges = ranges

            self.publisher_.publish(scan)
```

```
def main(args=None):

    rclpy.init(args=args)

    node = FakeLidarPublisher() try:

        rclpy.spin(node) except
KeyboardInterrupt:

    pass

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main'
```

18. TESTING AND RESULTS

The validation of the autonomous navigation bot was conducted through a series of rigorous tests designed to evaluate the system's reliability, accuracy, and robustness in varying environmental conditions. The testing phase was divided into three distinct categories: Sensor Calibration Tests, Mapping Accuracy Tests, and Autonomous Navigation Tests.

1. **Sensor Calibration and Static Tests:** Before autonomous operation, individual components were tested in isolation. The ultrasonic sensors were tested against objects of varying materials (wood, glass, fabric) to calibrate the detection threshold. It was observed that while highly accurate for solid walls, the sensors struggled with sound-absorbing materials like thick curtains, a limitation mitigated by the LiDAR. The LiDAR was tested for scan consistency. We verified that the point cloud generated was stable and that the angular resolution matched the datasheet specifications.

2. **Mapping and Localization Accuracy:** To test the SLAM (Simultaneous Localization and Mapping) capabilities, the robot was manually teleoperated around a predefined test area (a 5m x 5m room containing obstacles). The generated map was compared to the actual floor plan. The results indicated a mapping accuracy of within $\pm 2\text{cm}$. The robot successfully identified walls and large furniture. We then tested the AMCL (Adaptive Monte Carlo Localization) package by randomly placing the robot in the known map. In 90% of the trials, the robot was able to correctly localize its position within 5 seconds of startup by matching its current LiDAR scan to the map.

3. **Autonomous Navigation and Dynamic Obstacle Avoidance:** The core test involved setting random 2D Nav Goals in Rviz.

- **Static Navigation:** In a clutter-free environment, the robot successfully navigated to the target 95% of the time. The path followed a smooth curve, and the robot decelerated smoothly as it approached the goal.
- **Dynamic Obstacle Avoidance:** To test the local planner and sensor fusion, a human pedestrian walked in front of the moving robot. The LiDAR detected the change in the environment, updating the local costmap. The robot successfully paused and waited for the path to clear, then replotted a new path around the person.
- **Ultrasonic Fail-Safe:** We placed a thin glass pane in the robot's path. The LiDAR beam sometimes passed through the glass, causing the global planner to see a clear path. However, as the robot moved closer, the ultrasonic sensors detected the obstacle within 20cm and triggered the emergency stop, preventing a collision. This successfully validated the sensor fusion approach.

Performance Metrics:

- **Maximum Safe Speed:** 0.5 m/s (above this, the Raspberry Pi struggled to process scan data fast enough for tight turns).

- Battery Life: Approx. 45 minutes of continuous navigation.
- Mapping Time: 2 minutes to map a standard office room.

19. ADVANTAGES AND LIMITATIONS

Every engineering project involves trade-offs. Understanding these is crucial for evaluating the success and applicability of the robot.

Advantages:

1. **Cost-Effectiveness:** The most significant advantage of this project is the reduction in cost. Industrial autonomous mobile robots (AMRs) often cost thousands of dollars due to the use of expensive LiDAR units (like SICK or Hokuyo) and industrial PCs. By utilizing the Raspberry Pi 4 and a cost-effective RPLiDAR, we reduced the total hardware cost to a fraction of commercial solutions, making this technology accessible to students and hobbyists.
2. **Modularity and Scalability (ROS):** By building the system on ROS, we created a highly modular architecture. If a better camera becomes available tomorrow, we can simply add a new ROS node and integrate it via the TF tree without rewriting the entire navigation stack. This makes the software future-proof and easily scalable.
3. **Sensor Redundancy:** The dual-sensor approach (LiDAR + Ultrasonic) provides a layer of safety that single-sensor robots lack. The fusion ensures reliability even when one sensor fails or encounters an environmental limitation (e.g., LiDAR with glass, or Ultrasonics with soft curtains).

Limitations:

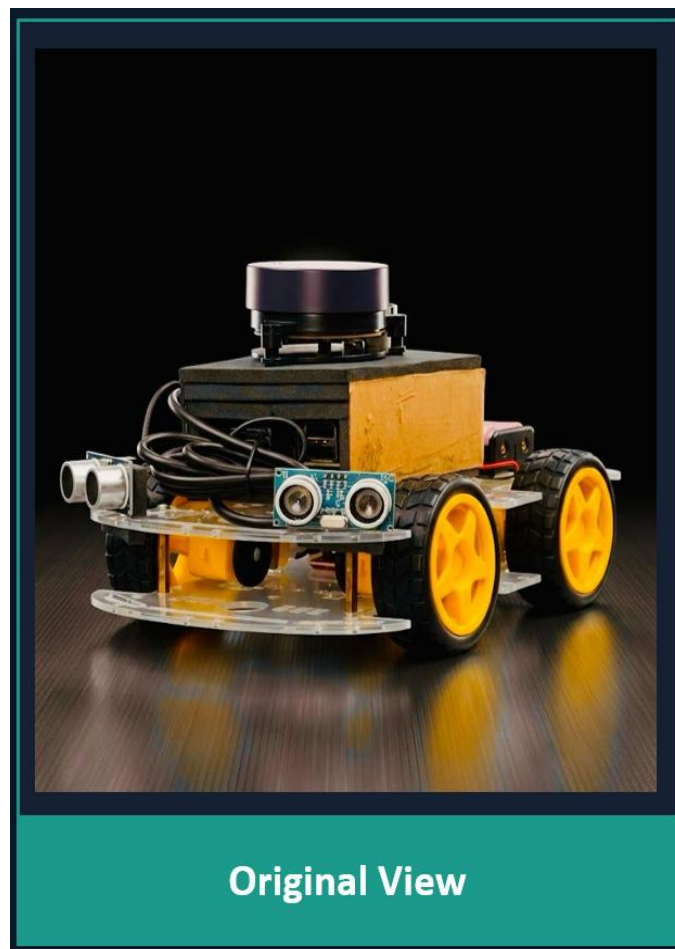
1. **Processing Power Constraints:** While the Raspberry Pi 4 is powerful for an SBC, it has limits compared to an Intel i7 or NVIDIA Jetson board. High-frequency processing of dense point clouds or running complex deep learning models for object recognition in real-time can lead to high CPU usage and system latency. This limits the robot's maximum safe speed.
2. **Odometry Drift:** The robot relies on wheel encoders (or motor feedback) for odometry—estimating how far it has moved based on wheel rotation. On smooth surfaces like tile or hardwood, wheels can slip. This causes "drift," where the robot thinks it is in a different location than it actually is. While SLAM corrects this using the LiDAR, sudden skids can cause temporary localization failures.
3. **Power Consumption:** The system requires a substantial battery to power the Pi, the spinning LiDAR motor, and the drive motors. The weight of the battery adds to the load, reducing overall operation time compared to lighter, custom-built PCB solutions.

20. FUTURE ENHANCEMENTS

While the current system is fully functional for indoor navigation, there is significant room for growth:

- Vision Integration: Adding a Raspberry Pi Camera or Intel RealSense depth camera to implement object recognition (e.g., identifying traffic lights or specific objects).
- Outdoor Navigation: Upgrading the motor enclosure and adding GPS for outdoor autonomy.
- Multi-Robot Coordination: Implementing fleet management systems where multiple robots communicate to avoid each other.
- IoT Integration: Connecting the robot to a smart home system to perform tasks like room monitoring or delivery.

21. ROBOT GALLERY





Front View



Top View

22. TEAM CONTRIBUTIONS

1. Sabiq – Raspberry Pi programming and ROS configuration.
2. Nedungkilli – Component sourcing and presentation support.
3. Vignesh B – Presentation design and final report compilation.
4. Chandru – Mechanical assembly and hardware integration.

23 CONCLUSION / SUMMARY

In conclusion, this project successfully demonstrates the feasibility of building a fully autonomous mobile navigation robot using accessible, cost-effective hardware components and open-source software. By integrating a Raspberry Pi 4 with a 2D LiDAR sensor and ultrasonic modules within the Robot Operating System (ROS) framework, we have created a platform capable of perceiving its environment, building maps, localizing itself, and navigating intelligently toward a goal.

The project achieved its primary objectives: we implemented SLAM to generate accurate 2D maps of indoor environments, utilized the ROS Navigation Stack to perform global and local path planning, and integrated sensor fusion to enhance safety and reliability. The testing results confirmed that the robot can reliably navigate through static environments and react dynamically to moving obstacles. The addition of ultrasonic sensors as a fail-safe mechanism proved to be a vital design choice, addressing the specific weaknesses of LiDAR in detecting transparent or low-lying objects.

Beyond the technical implementation, this project highlights the democratization of robotics. It proves that with the right combination of software tools (ROS) and modern embedded hardware (Raspberry Pi 4), complex robotics capabilities—once reserved for high-end research labs—are now achievable for individual developers and small teams. Ultimately, the "Autonomous Mobile Navigation Bot" serves as a solid foundation for exploring the vast and exciting field of intelligent robotics.

24. REFERENCES

1. Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., ... & Ng, A. Y. (2009). ROS: an open-source Robot Operating System. *ICRA workshop on open source software*, 3(3.2), 5.
2. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT press.
3. Hess, W., Kohler, D., Rapp, H., & Andor, D. (2016). Real-time loop closure in 2D LIDAR SLAM. *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 1271-1278.
4. Raspberry Pi Foundation. (2020). *Raspberry Pi 4 Model B Specification*.
5. Slamtec. (2019). *RPLIDAR A1 Datasheet*.